

# Corrigé – Récursivité

## Exercice 1

La factorielle  $n!$  d'un nombre entier  $n \geq 1$  est définie par  $n! = 1 \times 2 \times 3 \times \dots \times n$ .

|    |      |   |   |   |    |     |     |      |
|----|------|---|---|---|----|-----|-----|------|
| 1. | $n$  | 1 | 2 | 3 | 4  | 5   | 6   | 7    |
|    | $n!$ | 1 | 2 | 6 | 24 | 120 | 720 | 5040 |

2.  $n \times (n - 1)!$

3. (a) À l'exécution, on ne sort jamais de cette fonction car elle s'appelle à l'infini. Il faudrait une condition pour en sortir à un moment pertinent.

```
(b)1 def factorielle(n):  
    2     if n == 1:  
    3         return 1  
    4  
    5     return n*factorielle(n-1)
```

- (c) Dans le corps du script :

```
assert factorielle(1) == 1  
assert factorielle(2) == 2  
assert factorielle(3) == 6  
assert factorielle(4) == 24  
assert factorielle(5) == 120  
assert factorielle(6) == 720  
assert factorielle(7) == 5040
```

- (d) 1 appel initial puis 3 appels récursifs sont réalisés lors de l'appel `factorielle(4)`. Il y a donc 4 appels de la fonction en tout. On peut le vérifier en faisant un `print n` en tout début du corps de la fonction.

## Exercice 2

### Carrés imbriqués

```
1  # IMPORTS  
2  
3  from turtle import *  
4  from math import sqrt  
5  
6  # FONCTIONS  
7  
8  def carre(cote):  
9      for _ in range(4):  
10          forward(cote)  
11          right(90)  
12
```

```

13 def carres_imbriques(cote):
14     if cote < 1:
15         return
16
17     # tracer le carré
18     carre(cote)
19
20     # déplacement
21     forward(cote/2)
22     right(45)
23
24     # appel récursif
25     carres_imbriques(cote/sqrt(2))
26
27 # SCRIPT
28
29 setup(800, 600)
30 # speed(10)
31
32 carres_imbriques(200)
33
34 # boucle maintenant la fenêtre ouverte
35 mainloop()

```

## Triangles imbriqués

```

1  # IMPORTS
2
3  from turtle import *
4  from math import sqrt
5
6  # FONCTIONS
7
8  def triangle(cote):
9      for _ in range(3):
10         forward(cote)
11         right(120)
12
13 def triangles_imbriques(cote):
14     if cote < 1:
15         return
16
17     # tracer le triangle
18     triangle(cote)
19
20     # déplacement
21     forward(cote/2)
22     right(60)
23
24     # appel récursif
25     triangles_imbriques(cote/2)

```

```

26
27  # SCRIPT
28
29  setup(800, 600)
30  # speed(10)
31
32  triangles_imbriques(200)
33
34  # boucle maintenant la fenêtre ouverte
35  mainloop()

```

## Cercles et carrés imbriqués

```

1  # IMPORTS
2
3  from turtle import *
4  from math import sqrt
5
6  # FONCTIONS
7
8  def carre(cote):
9      for _ in range(4):
10         forward(cote)
11         right(90)
12
13  def carre_blanc(cote):
14      begin_fill()
15      color("white", "white")
16      carre(cote)
17      end_fill()
18
19  def disque_gris(cote):
20      begin_fill()
21      color("gray", "gray")
22      circle(cote)
23      end_fill()
24
25  def cercles_carres_imbriques(cote):
26      if cote < 1:
27         return
28
29      # tracer le carré blanc
30      carre_blanc(cote)
31
32      # déplacement position initiale cercle
33      right(90)
34      forward(cote/2)
35
36      # trace le cercle gris
37      disque_gris(cote/2)
38

```

```

39     # déplacement
40     left(135)
41
42     # appel récursif
43     cercles_carres_imbriques(cote/sqrt(2))
44
45 # SCRIPT
46
47 setup(800, 600)
48 # speed(10)
49
50 cercles_carres_imbriques(200)
51
52 # boucle maintenant la fenêtre ouverte
53 mainloop()

```

### Exercice 3

On considère la fonction récursive fibo ci-dessous, qui prend en argument un nombre entier.

```

1 def fibo(n):
2     if n<=1: # condition d'arrêt
3         return n
4     return fibo(n-1)+fibo(n-2)

```

1. 8
2. 25
3. 144

```

4.1 def fibo_iteratif(n):
5     # on règle les deux premiers termes
6     if n <= 1:
7         return n
8
9     n_moins_1 = 1
10    n_moins_2 = 0
11
12    for _ in range(2, n+1):
13        total = n_moins_1 + n_moins_2
14        n_moins_2 = n_moins_1 # attention à l'ordre des instructions !
15        n_moins_1 = total
16
17    return total

```

### Exercice 4

```

1 # FONCTIONS
2 def est_palindrome_ite(mot):
3     taille_mot = len(mot)
4     for indice in range(taille_mot//2):

```

```

5         if mot[indice] != mot[taille_mot-1-indice]:
6             return False
7     return True
8
9     # Autre version
10    def est_palindrome_ite_compact(mot):
11        return mot == mot[::-1]
12
13    def est_palindrome(mot):
14        if len(mot) <= 1: # si le mot est vide ou n'a qu'un seul caractère, c'est un
15            ↪ palindrome
16            return True
17
18        if mot[0] == mot[-1]: # comparaison du premier et du dernier caractère
19            return est_palindrome(mot[1:-1]) # on rappelle la fonction sur le mot sans
20                ↪ ses lettres extrêmes
21
22        return False
23
24    # SCRIPT
25    # batterie de tests pour vérifier nos fonctions (ne pas oublier de tester plusieurs
26    ↪ cas dont les cas de base !)
27    assert est_palindrome_ite("")
28    assert est_palindrome_ite("z")
29    assert est_palindrome_ite("été")
30    assert est_palindrome_ite("radar")
31    assert not est_palindrome_ite("Radar")
32    assert not est_palindrome_ite("spaghetti")
33
34    assert est_palindrome_ite_compact("")
35    assert est_palindrome_ite_compact("z")
36    assert est_palindrome_ite_compact("été")
37    assert est_palindrome_ite_compact("radar")
38    assert not est_palindrome_ite_compact("Radar")
39    assert not est_palindrome_ite_compact("spaghetti")
40
41    assert est_palindrome("")
42    assert est_palindrome("z")
43    assert est_palindrome("été")
44    assert est_palindrome("radar")
45    assert not est_palindrome("Radar")
46    assert not est_palindrome("spaghetti")

```

## Exercice 5

```
1  # FONCTIONS
2  def somme_it(lst):
3      total = 0
4      for element in lst:
5          total += element
6      return total
7
8  def somme_rec(lst):
9      if len(lst) == 0:
10         return 0
11
12         return lst[0] + somme_rec([lst[i] for i in range(1, len(lst))])
13
14  # SCRIPT
15  assert somme_it([]) == 0
16  assert somme_it([9999]) == 9999
17  assert somme_it([1, 1, 1, 1]) == 4
18  assert somme_it([1, 2, 3]) == 6
19
20  assert somme_rec([]) == 0
21  assert somme_rec([9999]) == 9999
22  assert somme_rec([1, 1, 1, 1]) == 4
23  assert somme_rec([1, 2, 3]) == 6
```

## Exercice 6

```
1  # FONCTIONS
2  def rendu_glouton(a_rendre, pieces):
3      """ rendu_glouton(int, int, list) -> list
4          a_rendre : la somme d'argent à rendre
5          pieces : un tableau de pièces possibles à rendre """
6      piece_max = pieces[0]
7
8      if a_rendre == 0:
9          return []
10
11      if a_rendre >= piece_max:
12          return [piece_max] + rendu_glouton(a_rendre - piece_max, pieces)
13
14      return rendu_glouton(a_rendre, [pieces[i] for i in range(1, len(pieces))])
15
16  # SCRIPT
17  valeurs_possibles = [100, 50, 20, 10, 5, 2, 1]
18
19  assert rendu_glouton(67, valeurs_possibles) == [50, 10, 5, 2]
20  assert rendu_glouton(291, valeurs_possibles) == [100, 100, 50, 20, 20, 1]
21  # si on ne dispose pas de billets de 100 :
22  assert rendu_glouton(291, [valeurs_possibles[i] for i in range(1, len(pieces))]) ==
     ↪ [50, 50, 50, 50, 50, 20, 20, 1]
```